Transacciones

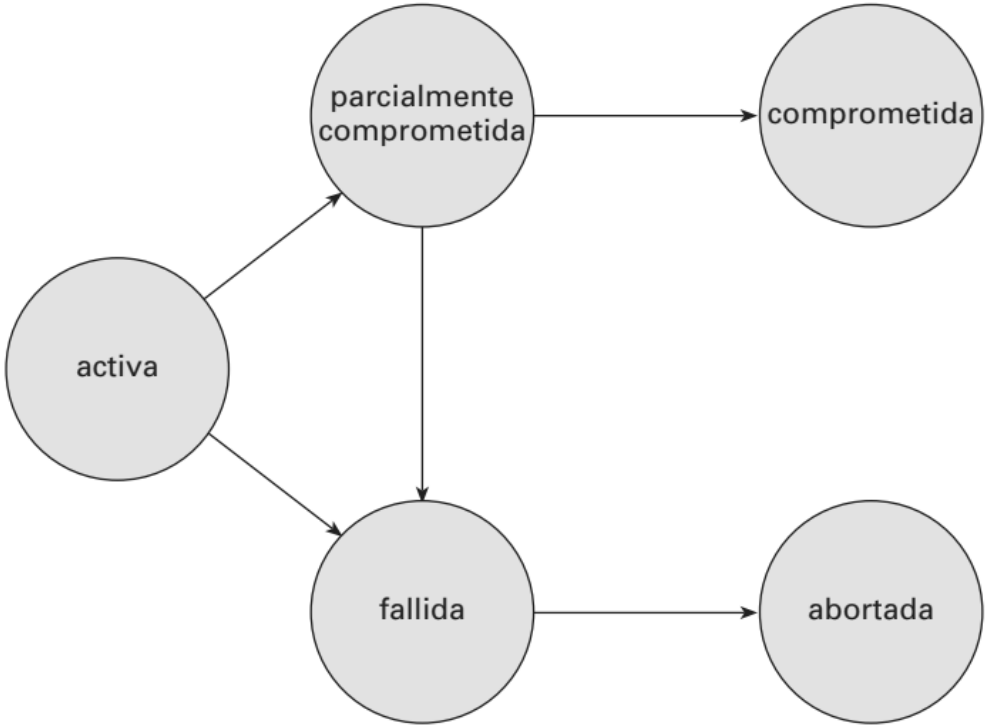
- ACID
- Estados de una transacción
- Start Transaction, Commit, Rollback
- Declare exit handler for sqlexception
- Ejercicios

Transacciones

Una transacción hace referencia a un conjunto de operaciones que forman una única unidad lógica de trabajo. Ejemplo: la transferencia de dinero de una cuenta a la otra, donde se considera la actualización de ambas cuentas.

Es importante en una transacción que se cumpla el **principio ACID**

Estados de una transacción



- **Activa**, el estado inicial; la transacción permanece en este estado durante su ejecución.
- **Parcialmente comprometida**, después de ejecutarse la última instrucción.
- **Fallida**, tras descubrir que no puede continuar la ejecución normal.
- **Abortada**, después de haber retrocedido la transacción y restablecido la base de datos a su estado anterior al comienzo de la transacción.
- **Comprometida**, tras completarse con éxito.

Una transacción se dice que ha **terminado** si se ha **comprometido** o bien se ha **abortado**.

Estados de una transacción

Comandos Claves

- **START TRANSACTION** (Inicio): Marca el punto de partida del bloque transaccional. Le indica al motor que las siguientes consultas (INSERT, UPDATE, DELETE) deben ejecutarse en memoria de forma temporal sin guardarse definitivamente aún.
- **COMMIT** (Confirmación): Consolida y guarda de forma permanente en disco todos los cambios realizados durante la transacción. Cierra el bloque con éxito y hace que los datos sean visibles para el resto de los usuarios.
- **ROLLBACK** (Deshacer): Cancela la transacción y revierte absolutamente todas las operaciones del bloque, *devolviendo la base de datos al estado exacto que tenía antes del START TRANSACTION*.

```
1   DELIMITER //
2   CREATE PROCEDURE sp_registrar_pago_con_estados(
3       IN  p_customer_id INT,
4       IN  p_staff_id INT,
5       IN  p_rental_id INT,
6       IN  p_amount DECIMAL(5,2),
7       OUT p_resultado VARCHAR(100)
8   )
9   BEGIN
10      START TRANSACTION;
11      IF p_amount <= 0 THEN
12          ROLLBACK;
13          SET p_resultado = 'ESTADO FINAL: ABORTADA (Monto menor o igual a cero)';
14      ELSE
15          INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
16          VALUES (p_customer_id, p_staff_id, p_rental_id, p_amount, NOW());
17      COMMIT;
18          SET p_resultado = 'ESTADO FINAL: COMPROMETIDA (Operación exitosa)';
19      END IF;
20  END //
21
22  DELIMITER ;
```

Ejercicio

1. Tomando el SP recién visto generar los siguientes casos de uso:
    - Un caso comprometido
    - Un caso abortado
    - Un caso donde no exista el cliente que se ingresa

```
DELIMITER //
CREATE PROCEDURE sp_registrar_pago_con_estados(
    IN  p_customer_id INT,
    IN  p_staff_id INT,
    IN  p_rental_id INT,
    IN  p_amount DECIMAL(5,2),
    OUT p_resultado VARCHAR(100)
)
BEGIN
    START TRANSACTION;
    IF p_amount <= 0 THEN
         ROLLBACK;
        SET p_resultado = 'ESTADO FINAL: ABORTADA (Monto menor o igual a cero)';
    ELSE
        INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
        VALUES (p_customer_id, p_staff_id, p_rental_id, p_amount, NOW());
    COMMIT;
        SET p_resultado = 'ESTADO FINAL: COMPROMETIDA (Operación exitosa)';
    END IF;
END //

DELIMITER ;
```

2. Cada mesa deberá tomar el SP creado la clase anterior, agregarle los comandos necesarios para contemplar los estados y probar con
    - Un caso comprometido
    - Un caso abortado
    - Un caso donde no exista uno de los parámetros ingresados

Estados de una transacción

Captura de errores

Si en medio de la transacción una sentencia falla (por ejemplo, pasaste **un customer_id que no existe**)....

MySQL interrumpe el flujo normal, entra al BEGIN...END del manejador, ejecuta el ROLLBACK para limpiar la memoria, asigna el mensaje de error al parámetro de salida y **finaliza el procedimiento ordenadamente**.

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
     ROLLBACK;
    SET p_resultado = 'ESTADO FINAL: ABORTADA (Fallo por error de SQL/FK)';
END;
```

- **DECLARE ... HANDLER FOR:** Define un "escuchador" que estará atento a lo que ocurra dentro del bloque BEGIN...END.
- **SQLEXCEPTION:** Es la condición global que lo activa. Captura **cualquier error técnico de SQL** (violación de clave foránea, intento de insertar un duplicado en PRIMARY KEY, valor NULL en columna obligatoria, etc.).
- **EXIT:** Es el comportamiento a seguir al capturar el error. Cancela de inmediato la ejecución del resto del procedimiento

Ejercicio

3. Agregar la declaración de la excepción al código del SP “sp_registrar_pago_con_estados” para probar cómo funciona

4. Agregarlo al caso de cada mesa

```
CREATE PROCEDURE sp_registrar_pago_con_estados(
    IN  p_customer_id INT,
    IN  p_staff_id INT,
    IN  p_rental_id INT,
    IN  p_amount DECIMAL(5,2),
    OUT p_resultado VARCHAR(100)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
         ROLLBACK;
        SET p_resultado = 'ESTADO FINAL: ABORTADA (Fallo por error de SQL/FK)';
    END;
    START TRANSACTION;
    IF p_amount <= 0 THEN
         ROLLBACK;
        SET p_resultado = 'ESTADO FINAL: ABORTADA (Monto menor o igual a cero)';
    ELSE
        INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
        VALUES (p_customer_id, p_staff_id, p_rental_id, p_amount, NOW());
    COMMIT;
        SET p_resultado = 'ESTADO FINAL: COMPROMETIDA (Operación exitosa)';
    END IF;
END //

```

Ejercicio – TP Integrador

- Pensar SP para una transacción principal de su BD
  - Alta de un movimiento
  - Modificación
  - Consulta pero con una función de por medio
  - Baja de un movimiento (pero manteniendo la consistencia de los datos)